CSC490 Assignment A2

Yibing Ju (1009382829), Cynthia Zhou (1009028918), Jonathan Chen (1008849310), Mark Noge (1008992299)

1 Part One: Aspirational Datasets

1. **Natural Language to Structured Event Dataset**
   A dataset like this would allow the training of a model to parse casual user input into formal calendar events. A possible schema could look like:

   { "user_input": "schedule 5 pm tmr for shopping", "parsed_output": { "event_name": "Shopping", "start_time": "2024-01-15T17:00:00", "end_time": "2024-01-15T18:00:00", "duration_minutes": 60, "location": null, "participants": [], "priority": "medium", "category": "personal_errands" }, "user_context": { "timezone": "America/New_York", "current_datetime": "2024-01-14T14:30:00" } }

2. **Longitudinal Schedule Patterns with User Demographics**
   This dataset would allow a model to understand scheduling behaviours across different types of users, enabling it to make more intelligent and accurate decisions. A possible schema could look like:

   { "user_id": "user_12345", "demographics": { "age_range": "25-34", "occupation": "software_engineer", "location": "urban", "household_type": "single", "work_style": "hybrid" }, "schedule_snapshot": [{ "date": "2024-01-15", "events": [{ "time": "09:00-17:00", "type": "work", "location": "office" }, { "time": "18:30-19:30", "type": "exercise", "location": "gym" }], "metrics": { "total_scheduled_hours": 9.5, "fragmentation_score": 0.3, "work_life_balance_ratio": 0.65 } }], "temporal_period": "6_months" }

3. **Event Taxonomy with Rich Attributes**
   Having this dataset would enable the model to categorize the properties of different event types with more accuracy. A possible schema could look like:

   { "event_id": "evt_grocery_shopping", "event_name": "Grocery Shopping", "categories": { "primary": "personal_errands", "secondary": ["household_management", "essential_tasks"] }, "attributes": { "typical_duration_minutes": 45, "duration_range": [30, 90], "flexibility_score": 0.8, "preferred_time_windows": ["morning_weekend", "evening_weekday"], "travel_time_required": true, "preparation_time_minutes": 5, "energy_level_required": "medium", "can_be_batched_with": ["pharmacy_pickup", "post_office"], "conflicts_with": ["deep_work", "meetings"], "optimal_frequency": "weekly" }, "context_variations": [{ "condition": "has_children", "duration_modifier": 1.3,

"preferred_times": ["weekend_morning"] }] }

4. **Conflict Resolution & User Preference Dataset**
   A dataset like this would teach the model how users handle scheduling conflicts and make trade off decisions based on their personal values and priorities. A possible schema could look like:

   { "scheduled_event": { "event_id": "evt_12345", "name": "Gym Workout", "scheduled_time": "2024-01-15T18:00:00", "duration_planned": 60 }, "actual_outcome": { "completed": true, "actual_start_time": "2024-01-15T18:15:00", "actual_duration": 45, "user_satisfaction": 4, "notes": "Felt rushed due to meeting running late" }, "context": { "preceding_event": { "name": "Client Call", "ended_late_by_minutes": 20 }, "day_metrics": { "total_scheduled_hours": 11, "back_to_back_meetings": 4, "user_stress_level": "high" } }, "learned_patterns": { "buffer_needed_after_client_calls": 30, "gym_completion_rate_when_overbooked": 0.4 } }

5. **Historical Event Completion & Behavior Dataset**
   A dataset like this would provide historical records of scheduled events and their actual outcomes, allowing the model to learn realistic scheduling patterns and predict event completion likelihood. A possible schema could look like:

   { "user_id": "user_12345", "historical_events": [{ "event_id": "evt_001", "event_type": "gym_workout", "scheduled_time": "2024-01-15T18:00:00", "scheduled_duration": 60, "actual_start_time": "2024-01-15T18:15:00", "actual_duration": 45, "completion_status": "completed", "day_context": { "total_scheduled_hours": 11, "preceding_event_delay_minutes": 20, "back_to_back_meetings_count": 4 } }, { "event_id": "evt_002", "event_type": "gym_workout", "scheduled_time": "2024-01-22T19:00:00", "scheduled_duration": 60, "actual_start_time": null, "actual_duration": null, "completion_status": "skipped", "day_context": { "total_scheduled_hours": 12, "preceding_event_delay_minutes": 45, "back_to_back_meetings_count": 6 } }], "completion_patterns": { "gym_workout": { "overall_completion_rate": 0.65, "completion_rate_when_scheduled_after_6pm": 0.45, "completion_rate_when_busy_day": 0.30, "average_delay_minutes": 12 } } }

2 Part Two: Reality Check

2.1 Dataset Sample Table

Relevant Item	Description	Commentary

Enron Email Dataset https://www.cs.cmu.edu/~enron/	Contains ~500K emails from Enron employees including calendar invites, meeting requests, and scheduling-related correspondence.	Can extract natural language scheduling patterns, meeting coordination examples, and understand how people communicate about scheduling in professional contexts
TimeUse Dataset (ATUS) https://www.bls.gov/tus/	American Time Use Survey - detailed time diary data showing how Americans spend their time across demographics	Provides realistic time allocation patterns by occupation, age, household type. Crucial for understanding typical scheduling behaviors across user segments
Reddit r/productivity & r/scheduling https://www.reddit.com/r/productivity/	Thousands of posts/comments about calendar management, scheduling conflicts, productivity tips, and time blocking strategies	Natural language corpus showing how people describe scheduling problems and preferences. Useful for training conversational understanding and identifying common pain points. We can scrape this data.
Meetup.com Event Data https://www.meetup.com/	Public event data including event names, times, durations, locations, categories (tech, fitness, social, etc.), and attendance patterns	Real-world event taxonomy with typical durations and time preferences. Helps build event attribute database (Dataset #3) - e.g., "networking events typically last 2 hours, occur evenings". Can use Meetup's API.
OpenStreetMap POI Data https://www.openstreetmap.org/#map=8/43.241/-78.673	Location data for venues (restaurants, gyms, offices, etc.) with addresses and coordinates	Enrich events with location context. Calculate travel times between events, suggest locations based on event type (Dataset #3 enhancement)

Microsoft BA-Calendar Dataset https://huggingface.co/datasets/microsoft/ba-calendar	Synthetic dataset for training scheduling assistants. Contains multi-participant availability schedules, meeting constraints (duration, buffer times, priority, time windows), and natural language scheduling queries.	Directly applicable for multi-party scheduling and conflict resolution (Dataset #4). Provides examples of constraint handling, availability parsing, and finding optimal time slots. Shows realistic meeting parameters and demonstrates how to structure scheduling queries for AI models.
Natural Plan - Calendar Scheduling https://github.com/google-deepmind/natural-plan	Benchmark dataset with 1,000 calendar scheduling scenarios for multi-participant meetings. Includes existing schedules, constraints (duration, availability, time windows), and Google Calendar API outputs. Varies by number of participants (3-7) and days (1-5).	Provides realistic multi-party scheduling scenarios with natural language queries and structured outputs (Datasets #1 and #4). Includes evaluation metrics to benchmark model performance. Shows real-world constraint satisfaction problems our AI needs to solve.
Events-Scheduling Dataset https://huggingface.co/datasets/anakin87/events-scheduling	Synthetic dataset for creating optimal schedules from event lists with priorities. Contains events with time slots, priority labels, and expected schedules that maximize weighted duration while avoiding conflicts.	Directly applicable to conflict resolution and priority-based scheduling (Dataset #4). Shows constraint satisfaction with overlapping events and formatted input/output pairs for training optimization models.
KVRET (Key-Value Retrieval Networks) https://nlp.stanford.edu/blog/a-new-multi-turn-multi-domain-task-oriented-dialogue-dataset/	3,031 multi-turn dialogues including 1,034 calendar scheduling conversations collected via Wizard-of-Oz method. Contains dialogue state annotations, slots, API calls, and grounded knowledge bases showing calendar states.	Provides real human conversations about calendar scheduling requests. Shows natural language patterns for scheduling tasks with multi-turn context (Dataset #1). Perfect for training conversational scheduling interfaces.

MTUS (Multinational Time Use Study) https://www.timeuse.org/mtus	2+ million time diary days from 100+ surveys across 25 countries (1965-2018). Contains harmonized activity data with 41-69 categories, demographics, timing, duration, and day-of-week patterns. Free registration required.	Provides large-scale real scheduling patterns by demographics (Dataset #2). Shows how people of different ages, occupations, and household types allocate time throughout their day. Can inform realistic event duration and timing patterns for your event taxonomy (Dataset #3).

Although we identified several potentially relevant public datasets, we do not plan to rely on them directly at this stage because none of them fully match our system's input, output format or product goals. Most datasets capture only fragments of the problem (e.g., dialogue data, time-use statistics, or synthetic scheduling scenarios), and would require significant preprocessing or assumptions before they could be used for model evaluation.

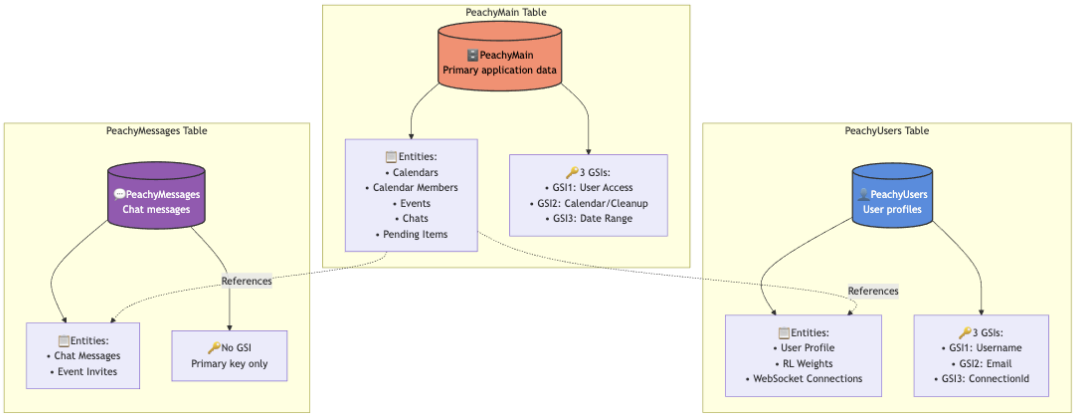
Instead, our immediate plan is to construct a small internal dataset tailored to our pipeline by generating examples that reflect our app's real usage patterns and schema. This will allow us to run basic evaluations of parsing accuracy and latency, and user-edit frequency using data that closely mirrors production inputs.

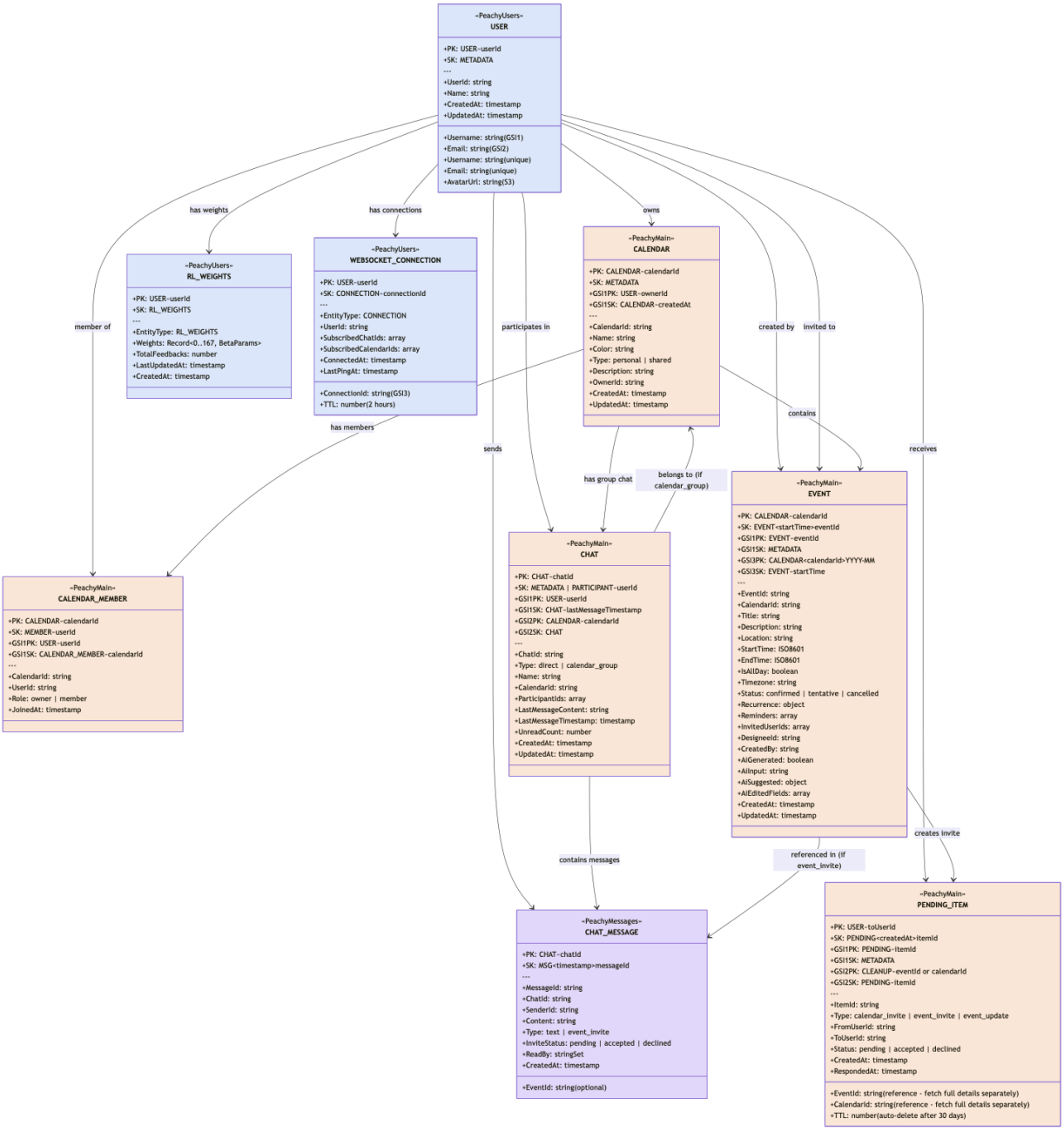
The external datasets we identified will still be valuable as reference sources or fallback training data. If the LLM does not meet our expectations in terms of accuracy or speed, these datasets may later be incorporated to augment training signals, benchmark performance, or identify missing scenario coverage.

3 Part Three: Data-processing pipelines

Data Schema

We use Amazon DynamoDB as the primary operational data store. The schema follows a single-table design for core application entities, using composite partition/sort keys to model relationships and support common query patterns. Separate tables are used only when entities have significantly different scaling, retention, or access characteristics.


PeachyMain Table
PeachyMain
Primary application data
Entities:
• Calendars
• Calendar Members
• Events
• Chats
• Pending Items
3 GSIs:
• GSI1: User Access
• GSI2: Calendar/Cleanup
• GSI3: Date Range
PeachyMessages Table
PeachyMessages
Chat messages
References
Entities:
• Chat Messages
• Event Invites
No GSI
Primary key only
PeachyUsers Table
PeachyUsers
User profiles
References
Entities:
• User Profile
• RL Weights
• WebSocket Connections
3 GSIs:
• GSI1: Username
• GSI2: Email
• GSI3: ConnectionId



The schema models relationships using key composition rather than joins:

- A User owns Calendars and creates Events
- A Calendar contains Events and has Members
- A Calendar may also contain a group Chat
- A Chat contains Messages
- An Event generates Pending Items (invites or updates)
- Pending Items reference the Event or Calendar that triggered them
- WebSocket connections belong to Users for real-time updates

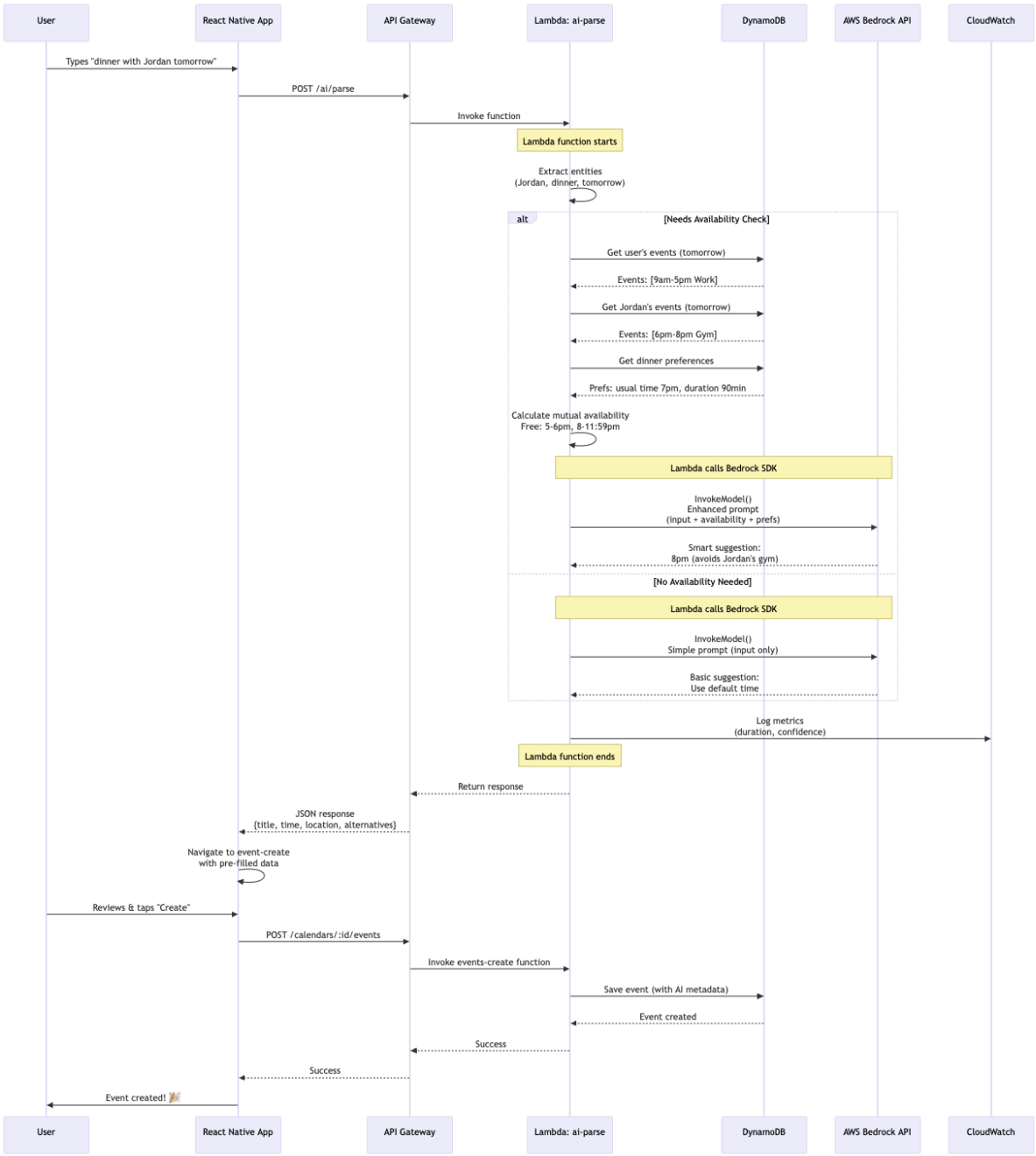
These relationships are resolved through partition keys, sort keys, and GSIs rather than relational joins, enabling scalable access patterns.

Full schema definition:

https://github.com/UofT-CSC490-W2026/Peachy-Infra/blob/dev/dynamodb-schema.md

Processing Pipeline

The Peachy system processes user scheduling requests through a real-time serverless pipeline that performs entity extraction, availability checking, AI suggestion generation, and structured event creation. The flow begins when a user submits a natural-language scheduling request from the mobile app. The request is sent through API Gateway to a Lambda function, which extracts entities and determines whether availability checking is required. When needed, the function queries DynamoDB for participant schedules and preference signals, then calls AWS Bedrock to generate a structured event suggestion. The response is returned to the client to pre-fill the event creation form, and once confirmed, the event is stored in DynamoDB with AI-related metadata.



To support evaluation of model performance, each stored event includes the fields AiGenerated, AiInput, and AiEditedFields. These attributes allow analysis of how frequently users accept AI suggestions versus modify them. High edit rates for specific fields (such as time, location, or title) indicate where the model underperforms and where prompt tuning or additional signals may be needed.

CloudWatch logs generated by the Lambda functions are also used for operational monitoring, including tracking latency, invocation success, and error rates.

Full pipeline documentation:
https://github.com/UofT-CSC490-W2026/Peachy-Infra/blob/dev/data-processing-pipeline.md

Code for an initial version of this pipeline

Frontend code: https://github.com/UofT-CSC490-W2026/Peachy/tree/dev

Backend code: https://github.com/UofT-CSC490-W2026/Peachy-Infra/tree/dev

The initial pipeline implementation is in the Peachy-Infra repository under the Lambda APIs:
https://github.com/UofT-CSC490-W2026/Peachy-Infra/tree/dev/lambda
The core ai parse lambda is in:
https://github.com/UofT-CSC490-W2026/Peachy-Infra/blob/dev/lambda/ai/parse/index.ts

This Lambda implements the AI event parsing pipeline end-to-end. It receives a user's natural-language request, decides whether the request requires an availability check (smart path) or can be handled without it (fast path), and conditionally queries DynamoDB for participant calendar data and RL preference weights. It then calls AWS Bedrock(Claude Haiku) using a structured prompt and tool-calling to return JSON event objects (e.g., title, startTime, endTime, location, invitees). The parsed event is returned to the frontend and used to pre-fill the event creation form.

All currently implemented backend APIs are wired to and exercised by the frontend flow.

Next Steps

- Complete the remaining API endpoints, including profile editing and additional event invitation management features.
- Implement the chat feature using real-time messaging via WebSocket (API Gateway WebSocket API).
- Test, evaluate, and improve the AI pipeline by measuring Bedrock parsing accuracy and refining prompts based on observed errors.
- Add LLM-generated category descriptions, automatically producing or refreshing calendar/category summaries when new events are added (this will be deprioritized if model performance is insufficient).
- Conduct user research and usability testing to validate terminology choices (e.g., *"Calendar" vs. "Category"*) and improve the overall scheduling experience.

4 Part Four: Infrastructure as Code Implementation

You can check out our IaaC repo here:
https://github.com/UofT-CSC490-W2026/Peachy-Infra/tree/dev

Our infrastructure is defined entirely in code using AWS CDK, organized as one stack per logical service: Database, Storage, Auth, Notifications, AI, Monitoring, and API. Each stack is defined in its own file under resources/ and instantiated from a single entry point (main.ts).

Environment-specific configuration (dev vs prod) is managed through a typed config object in resources/config.ts, which controls settings like deletion policies, Cognito domain prefixes, and feature flags — the same codebase deploys to both environments with --context env=dev|prod.

Data-bearing resources (DynamoDB tables, S3 buckets, Cognito user pools) use CDK's RemovalPolicy.RETAIN in production to ensure data survives infrastructure changes. Lambda functions are bundled with a shared layer (lambda/shared/) for common dependencies, and all IAM policies follow least-privilege via centralized utilities in iam.ts. Deployment and recovery are handled through scripts (scripts/deploy-prod.ps1, scripts/recover-prod.ps1) that validate credentials, synthesize templates, and orchestrate CloudFormation operations. A RUNBOOK.md documents disaster recovery procedures.

5 Part Five: Disaster Recovery Demonstration

Demo Video Link:
https://drive.google.com/file/d/1CtC6rEEdVXSkOw1AiT0IAu1PXj-nZCJ7/view?usp=sharing

Please Read Before Viewing:

This is an entirely uncut video, around 37 minutes long, to ensure it is very clear that no video editing was used to misinterpret how the disaster recovery process works.

As a result, many parts of the video were simply me sitting and waiting for certain commands to finish running (e.g. redeployment of the stacks). I will hence provide some timestamps should you find yourself wanting to revisit specific parts of the video during grading or skip portions of the video.

Timestamps:
11:45 - Destroy prod script runs
15:45 - Destroy prod script finishes
19:08 - Recovery script starts
29:20 - Recovery script finishes

Video Summary/Introduction:

This demo shows disaster recovery for our production AWS infrastructure. Our CDK codebase uses a RETAIN deletion policy on all data-bearing resources — DynamoDB tables, S3 buckets, and Cognito user pools. When stacks are destroyed, these resources survive as orphans in AWS. Our recovery script detects those orphaned resources through their unique identifiers and uses the CloudFormation Import API to adopt them back into new stacks, then runs a full CDK deploy to recreate stateless resources like Lambda functions and API Gateway. The result is a fully restored production environment with all user data, auth configuration, and stored files intact.

Summary of relevant files (in Peachy-Infra)

Documentation:
- RUNBOOK.md — Documents what survives deletion, what's lost, and step-by-step recovery procedures for both automated and manual recovery.

Infrastructure Configuration:
- resources/config.ts — Defines per-environment settings. Prod has retainDataOnDelete: true, which controls whether resources survive stack deletion.
- resources/database-stack.ts — Creates three DynamoDB tables (main, users, messages) with RETAIN policy in prod so table data is never lost.
- resources/storage-stack.ts — Creates the S3 media bucket with RETAIN policy, preserving uploaded files through stack deletions.
- resources/auth-stack.ts — Creates the Cognito user pool, app client, domain, and Google identity provider, all with RETAIN in prod to preserve user accounts.

Scripts:
- scripts/destroy-prod.ps1 — Runs cdk destroy --all to tear down all prod stacks. Retained resources survive as orphans in AWS.
- scripts/recover-prod.ps1 — Discovers orphaned resources via AWS CLI, builds CloudFormation import changesets to adopt them back into new
  stacks, then runs cdk deploy to recreate stateless resources like Lambdas and API Gateway.